Large Language Models

Supervised Fine-Tuning (SFT)

Teaching LLMs to Formulate Text & Follow Instructions

Yakov · September 2026

The LLM Alignment Pipeline

 Pre-training
<small>(Base Model)</small> → SFT
<small>(Instruction Model)</small> → RLHF / DPO
<small>(Human Preferences)</small>

- **Pre-training:** web-scale raw knowledge and language structure acquisition.
- **SFT:** teaching interaction formats, tasks, and system behaviors.
- **RLHF/DPO:** micro-tuning behavior to meet safety standards and human feedback.

Pre-training vs. SFT (The Mindset Shift)

  Base LLM — Text Completer Prompt: Write a python function for quicksort. Completion: Write a python function for mergesort.
Write a python function for binary search.
Write a python function for… Chaos pattern: treats the prompt as one item in a worksheet, not a request — keeps completing the list instead of answering it.   SFT LLM — Helpful Assistant Prompt: Write a python function for quicksort. Completion: Here is the quicksort implementation: <code>def quicksort(arr): ... return arr</code> Alignment pattern: transitions from pure raw completion to playing the persona of an assistant.

Prompt Engineering vs. SFT: The Core Tradeoff

 <table class="decode-table"> <thead> <tr><th>Metric</th><th>Prompt Engineering (Few-Shot)</th><th>Supervised Fine-Tuning (SFT)</th></tr> </thead> <tbody> <tr><td>Upfront Cost</td><td>$0 — immediate setup via API</td><td>High — data collection + GPU rent</td></tr> <tr> <td>Cost Per Request</td><td>High — paying for examples every time</td><td>Minimal — short, clean prompts</td></tr> <tr><td>Format Reliability</td><td>Weak — model can drift or forget rules</td> <td>Hardcoded — baked into model weights</td></tr> </tbody> </table> Optimization path: cheap, ad-hoc customization (prompting) trades off against low per-request cost and durable behavior (SFT) as usage scales.

The Anatomy of an Instruction Dataset

ChatML / OpenAI JSON template structure

```
[
  {"role": "system", "content": "You are a helpful coding assistant."},
  {"role": "user", "content": "How do I reverse a list in Python?"},
  {"role": "assistant", "content": "You can reverse a list using `list.reverse()` or slicing `[::-1]`."}
]
```

[Prompt] + [Assistant] → Loss The Gold Target — loss is calculated ONLY on the assistant response.

- **System:** defines boundaries/tone.
- **User:** input instructions.
- **Assistant:** the training target.

Quality Over Quantity: The LIMA Effect

 Curated LIMA (1k) ~high win rate Web Scraps (100k+) ~lower win rate   Definition: Human Win Rate Blind A/B tests where human experts compare two anonymous model outputs for the same prompt. If Model A wins 75/100 matchups, its win rate is 75%. LIMA (Less Is More for Alignment): a tiny set of 1,000 meticulously filtered, high-quality instruction responses consistently outperforms massive, multi-thousand-sample datasets full of noise and repetitive patterns.

Data Synthetic Generation (Self-Instruct)

 Teacher Model
<small>(e.g. GPT-4)</small> → Raw Prompt &
Response Generation → Filtering
<small>Heuristics & LLM-eval</small> → Student
Model
<small>SFT Engine</small> The student model's outputs can loop
back to seed the next generation round (dashed feedback in the original pipeline).

- **Pipeline:** seed tasks → generation loops → deduplication checks → target training.
- **Pitfall:** risk of data echo chambers — repeating systemic hallucination footprints of the teacher.

The Necessity of Chat Templates

The Challenge: Text Completion vs. Interaction

- **Base model behavior:** predicts the next word in a sequence. Without structure, it treats a prompt as a *continuation* of text (e.g., finishing your question).
- **The goal:** turn an open-ended generator into a controlled *dialogue agent*.

  Expectation (Controlled) <code>User: 4+4?</code>
<code>Assistant: 8</code>
<code>[STOP TOKEN]</code>   Reality (Uncontrolled — Role Bleeding) <code>User: 4+4?</code>
<code>Assistant: 8</code>
<code>User: And what is 5+5?</code>
<code>Assistant: 10</code>
<code>[STOP TOKEN]</code>

The Solution: Chat Templates

Enforcing boundaries with special tokens

 Model respects the structure <code>
<|im_start|>User: What is 4+4?<|im_end|></code>
<code><|im_start|>Assistant: 8<|im_end|></code>

- **Containment:** the model is confined to the marked scope.
- **Control:** the `<|im_end|>` tag serves as an explicit "Stop" signal for the tokenizer.

 Crucial technical step Tags must be registered as
Special Tokens to prevent fragmentation.
Fix: <code>tokenizer.add_special_tokens()</code> → <code>model.resize_token_embeddings()</code>.

Standard SFT Loss Function

 <code><|user|> What is 2+2? <|assistant|></code> <code>The answer is 4.</code>

$$\mathcal{L} = -\sum_{t=i}^{N} \log P(x_t \mid x_{<t})$$

 where i is the first token of the assistant's answer

- **Masked multi-turn loss:** only the assistant's response contributes to the loss.
- **Gradient flow:** backpropagation is constrained to target tokens, forcing the model to learn answer generation, not prompt replication.

Full Fine-Tuning vs. PEFT

  Full Fine-Tuning (FT) Δθ Δθ Δθ Δθ Δθ Δθ Δθ Δθ Δθ Δθ Δθ Δθ Δθ Modifies every single parameter. High risk of catastrophic forgetting. Massive VRAM requirements to maintain gradients and optimizer states.   PEFT (LoRA) Frozen + A B Strategy: train only a tiny fraction of parameters. Base model remains locked (99%+ frozen). Injects trainable matrices A, B: ΔW = AB.

Inside LoRA (Low-Rank Adaptation)

$$W_{\text{new}} = W_0 + \Delta W = W_0 + \frac{\alpha}{r}(B \times A)$$

  W₀ ∈ ℝ^{d×d} + A ∈ ℝ^{d×r} × B ∈ ℝ^{r×d}   <table class="decode-table"> <thead><tr><th>Method</th><th>Trainable Params / Layer</th></tr></thead> <tbody> <tr><td>Full FT</td><td>~16.8M (d=4096)</td></tr> <tr><td>LoRA (r=8)</td><td>~65K — ≈256× reduction</td></tr> </tbody> </table>

- **Assumption:** target adjustments ΔW feature highly reduced intrinsic rank.
- **Setup:** A initializes Gaussian; B initializes at zero → ΔW = 0 at step zero.

Memory Layout: Quantization & QLoRA

 FP32 FP16 NF4   4 Bytes 2 Bytes 0.5 Bytes

VRAM footprint for a standard 7B parameter model:

 <table class="decode-table"> <thead><tr><th>Precision Mode</th> <th>Data Format</th><th>Weights Footprint</th><th>Total VRAM for SFT</th></tr></thead> <tbody> <tr> <td>Full Precision</td><td>FP32</td><td>~28 GB</td><td>~112 GB (incl. states)</td></tr> <tr><td>Half Precision</td><td>FP16 / BF16</td><td>~14 GB</td><td>~40-50 GB</td></tr> <tr><td>QLoRA</td><td>NF4 (NormalFloat4)</td><td>~4.5 GB</td><td>< 8 GB (consumer-grade GPU)</td></tr> </tbody> </table>

Hyperparameters That Actually Matter

  Learning Rate   Cosine decay schedule — high → low.
~1e-4 for LoRA runs.
~2e-5 for full FT runs.
Magnitudes lower than pre-training rates.   Batch Size   Effective batch size: samples processed before updating weights (target: 32-128).
Gradient accumulation: sum gradients over mini-batches when GPU is too small, then update once.   Context Strategy   Padding: wastes tensor space via zero allocation.
Packing: chains short strings sequentially, separated by EOS tokens, to maximize sequence efficiency.

Fine-tuning Challenges: Overfitting & Forgetting

  1. Overfitting Model memorizes noise instead of patterns. Train loss keeps falling; validation loss falls, then turns back up — that turning point is where to stop. Solution: Early Stopping   2. Catastrophic Forgetting New skills override base general intelligence.   10% Web Tokens 90% SFT Dataset   Mitigation: Data Rehearsal

How to Evaluate an SFT Model?

  Benchmarks
MMLU, GSM8K, HumanEval
(rigid metrics / ground-truth match)   <div
class="bottleneck-title">LLM-as-a-Judge GPT-4 Evaluation, MT-Bench
(relative quality checks using strong models)   <div
class="bottleneck-title">Human Evaluation Side-by-side A/B testing
(gold standard verification; resource heavy)
Legacy text metrics (ROUGE / BLEU): mostly ineffective for abstract alignment tracking —
language answers change shape without affecting accuracy.